

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию
“Создание игры на юнити”

Выполнил:
Студент группы ИУ5-3 4Б
Шерстеникин Н.С.
Преподаватель:
Нардид А.Н.

Москва 2025

Задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Код скрипта PlayCont

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    public float speed = 5.0f; // устанавливаю константное значение скорости и
    других параметров движения при нажатии кнопки
    public float rotationSpeed = 120.0f;
    public float FlySpeed = 0.5f;
    public float MaxFlySpeed = 0.5f;

    private Rigidbody rb; // Получаю доступ к свойству тела игрока rigidbody

    // Start is called before the first frame update
    private void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    // Update is called once per frame
    void Update()
    {
    }

    private void FixedUpdate()
    {
    }
}
```

```

        float moveVertical = Input.GetAxis("Vertical");// Получаю изменение нажатия
кнопки
        Vector3 movement = transform.forward * moveVertical * speed *
Time.fixedDeltaTime;
        rb.MovePosition(rb.position + movement); // передаю посчитанную скорость
свойству тела игрока

        float turn = Input.GetAxis("Horizontal") * rotationSpeed *
Time.fixedDeltaTime;
        Quaternion turnRotation = Quaternion.Euler(0f, turn, 0f);
        rb.MoveRotation(rb.rotation * turnRotation);

        float moveUp = Input.GetAxis("Up");

        if (moveUp != 0f) {
            rb.useGravity = false; // отключаю воздействие гравитации на игрока для
избежания уменьшения скорости полёта при повторном взлёте в воздухе

            Vector3 flying = transform.up * moveUp * FlySpeed;
            rb.AddForce(flying, ForceMode.Force); // устанавливаю силу, тянущую
игрока вверх

            if (rb.linearVelocity.y > MaxFlySpeed) {
                rb.linearVelocity = new Vector3(rb.linearVelocity.x, MaxFlySpeed,
rb.linearVelocity.z); // Устанавливаю максимальную скорость полёта игрока
            }
        }
        else
        {
            rb.useGravity = true;
        }

        float Reset = Input.GetAxis("reset");

        if (Reset != 0f) {

            Vector3 alpha = new Vector3(0.5f, 0.5f, 0.5f);
            transform.position = alpha;
            transform.rotation = Quaternion.identity; // устанавливаю начальную
позицию для кнопки ресета
        }
    }
}

```

Код скрипта StarMovement

```

using UnityEngine;
using System;

public class StarMovement : MonoBehaviour
{
    public float rotationSpeed = 0.5f;
    public GameObject onCollectEffect;
    public bool isCollected = false;

    public System.Action<StarMovement> OnModified; // Может быть вызван ивент для
взаимодействия с другим скриптом
    public static event Action OnStarCollected;

    public void SetCollected(bool value)
    {
        if(isCollected != value)
        {
            isCollected = value;
        }
    }
}

```

```

        OnModified?.Invoke(this); // вызов ивента
    }
}

void Start()
{
}

// Update is called once per frame
void Update()
{
    transform.Rotate(0, rotationSpeed, 0); // задаю вращение звезды
}

private void OnTriggerEnter(Collider other) // Если какой-либо объект
соприкоснулся с телом звезды
{
    if (other.CompareTag("Player")) //Если с звездой соприкоснулся объект типа
    "Игрок"
    {
        // Может быть вызван ивент для взаимодействия с другим скриптом
        SetCollected(true);

        OnStarCollected?.Invoke();

        if (onCollectEffect != null)
            Instantiate(onCollectEffect, transform.position,
transform.rotation); // Вызываю эффект подбора звезды

        Destroy(gameObject); // Уничтожаю звезду
    }
}
}

```

Код скрипта collect

```

using UnityEngine;

public class collect : MonoBehaviour {
    // Start is called once before the first execution of Update after the
MonoBehaviour is created
    public bool collected;

    void Start()
    {
        foreach (Transform child in transform)
        {
            var childScript = child.GetComponent<StarMovement>(); //задаю скрипт
"Star movement" как дочерний для дальнейшего взаимодействия с ним
            if (childScript != null)
            {
                childScript.OnModified += HandleChildModified;
            }
        }
    }

    void HandleChildModified(StarMovement modifiedChild)
    {
        collected = true;
    }

    // Update is called once per frame
    void Update()
    {
    }
}

```

```

    {
    }

}

```

Код скрипта adding a star

```

using UnityEngine;

public class addingastar : MonoBehaviour {
    // Start is called once before the first execution of Update after the
    MonoBehaviour is
    public GameObject Collectible_Star;
    public GameObject VFX_Splatter;
    public GameObject ParentObject;
    int ZCoord;
    float YCoord;
    int XCoord;
    public Finale exmpl;

    void Start()
    {
    }

    // Update is called once per frame
    void Update()
    {
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            int ZCoord = UnityEngine.Random.Range(-8, 1);
            float YCoord = UnityEngine.Random.Range(0.5f, 2.5f);
            int XCoord = UnityEngine.Random.Range(-3, 3);

            Vector3 randomPos = new Vector3(XCoord, YCoord, ZCoord);

            GameObject newStar = Instantiate(Collectible_Star); //создаю новую звезду
            newStar.transform.position = randomPos; //задаю ей случайное положение
            newStar.transform.SetParent(ParentObject.transform); //приобщаю её к
            материнскому объекту для более удобного взаимодействия с интерфейсом, поиском звезды

            newStar.GetComponent<BoxCollider>().isTrigger = true; //задаю то, что она
            может быть триггером

            StarMovement StarScript = newStar.AddComponent<StarMovement>();
            StarScript.rotationSpeed = 0.5f;
            StarScript.onCollectEffect = VFX_Splatter;
            exmpl.maxStars += 1; //задаю её характеристики, скрипт и меняю
            глобальную переменную maxStars
        }
    }
}

```

Код скрипта Finale

```

using UnityEngine;

```

```

using System;

public class Finale : MonoBehaviour
{
    public int stars = 0;
    public int maxStars = 2;
    public static event Action OnFinale;

    void Start()
    {
        StarMovement.OnStarCollected += HandleStarCollected; //В случае, если звезда
        собрана, вызывается скрипт HandleStarCollected
    }

    void HandleStarCollected()
    {
        stars++;
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            if(stars == maxStars)
            {
                OnFinale?.Invoke();
            }
        }
    }

    void OnDestroy()
    {
        StarMovement.OnStarCollected -= HandleStarCollected;
    }
}

```

Код скрипта WinProcedure

```

using UnityEngine;
using System.Collections;
using UnityEngine.UI;

public class WinProcedure : MonoBehaviour
{
    [SerializeField] private float visibleDuration = 3f; // задаю время видимости
    надписи
    private Graphic[] uiGraphics;

    void Start()
    {
        uiGraphics = GetComponentsInChildren<Graphic>(true);

        SetVisible(false);

        Finale.OnFinale += HandleWin; // Если вызвано событие OnFinale из скрипта
        Finale, то активируется скрипт HanleWin
    }

    void HandleWin()
    {
        StartCoroutine>ShowAndHide()); // надпись становится видна
    }
}

```

```

private IEnumerator ShowAndHide()
{
    SetVisible(true); // надпись видна
    yield return new WaitForSeconds(visibleDuration); // ждём несколько секунд
    SetVisible(false); // надпись не видна
}

private void SetVisible(bool isVisible)
{
    if (uiGraphics != null)
    {
        foreach (var graphic in uiGraphics)
        {
            graphic.enabled = isVisible;
        }
    }
}

void OnDestroy()
{
    Finale.OnFinale -= HandleWin;
}
}

```

Работа приложения



